

20260917 - Tutorial (Cdiff gLV)

September 17, 2026

```
[23]: import itertools
import pandas as pd

import seaborn as sns
import scipy as sp
from scipy.stats import linregress, pearsonr
import time

import sys
import os

import matplotlib.pyplot as plt
import numpy as np

# Get the current working directory (where the script is run from)
current_dir = os.getcwd()

# Get the parent directory
parent_dir = os.path.dirname(current_dir)

# Add the parent directory to sys.path
sys.path.append(parent_dir)

from nsm.glv import *

# set plot parameters
params = {'legend.fontsize': 18,
          'figure.figsize': (8, 7),
          'axes.labelsize': 24,
          'axes.titlesize': 24,
          'axes.linewidth': 3,
          'xtick.labelsize': 20,
          'ytick.labelsize': 20,
          'xtick.labelsize': 20,
          'ytick.labelsize': 20,
          'svg.fonttype': 'none'}
plt.rcParams.update(params)
```

```
np.random.seed(12345)
```

```
[ ]:
```

```
[24]: # import data
df = pd.read_csv("../data/sulaiman_data_fmt.csv")
df.head()
```

```
[24]:
```

	Treatments	Time	BT	BV	CH	BU	CS	\
0	BT-BU-BV-CA-CS-DP	0	0.001667	0.001667	0.0	0.001667	0.001667	
1	BT-BU-BV-CA-CS-DP	12	1.116771	0.190538	0.0	0.114889	0.127689	
2	BT-BU-BV-CA-CS-DP	24	1.193564	0.044796	0.0	0.097576	0.297728	
3	BT-BU-BV-CA-DP	0	0.002000	0.002000	0.0	0.002000	0.000000	
4	BT-BU-BV-CA-DP	12	1.116553	0.312622	0.0	0.063024	0.000000	

	CA	DP	DSM	MS001	MS008	MS014
0	0.001667	0.001667	0.0	0.0	0.0	0.0
1	0.002910	0.021037	0.0	0.0	0.0	0.0
2	0.077205	0.031381	0.0	0.0	0.0	0.0
3	0.002000	0.002000	0.0	0.0	0.0	0.0
4	0.001057	0.018078	0.0	0.0	0.0	0.0

```
[25]: # define species and mediator names
species = ['BT', 'BV', 'CH', 'BU', 'CS', 'CA', 'DP', 'DSM', 'MS001', 'MS008', 'MS014']
mediators = []
inputs = []
```

```
[26]: # init model
model = gLV(dataframe=df,
            species=species,
            mediators=mediators,
            inputs=inputs,
            nu2=.001, sigma2=.01)

# init params
model.init_params(sample=False)
print(f"Number of parameters: {model.n_params}")
```

Number of parameters: 132

```
[27]: print(f"Total parameters: {model.n_params}")
t0 = time.time()
model.fit_posterior_EM(lr=.001,
                      n_sample=1,
                      n_sample_hypers=100,
                      max_epochs=10000,
                      patience=1, max_iterations=10)
```

```
print("Elapsed time: {:.3f}s".format(time.time()-t0))
```

Total parameters: 132

Updating posterior...

Epoch 0, ELBO: -44274.723, Slope: 1.000

encountered 0 nans

KeyboardInterrupt Traceback (most recent call last)

Cell In[27], line 3

```
1 print(f"Total parameters: {model.n_params}")
2 t0 = time.time()
----> 3 model.fit_posterior_EM(lr=.001,
4                               n_sample=1,
5                               n_sample_hypers=100,
6                               max_epochs=10000,
```

File g:

```
↪ \True_FYP\01_Project_Overview\Paper\Physics-constrainedneuralordinarydifferenialequationm
↪ py:483, in gLV.fit_posterior_EM(self, lr, n_sample, max_epochs,
↪ n_sample_hypers, patience, max_iterations)
478 def fit_posterior_EM(self, lr=1e-3, n_sample=1, max_epochs=100000,
↪ n_sample_hypers=100, patience=1,
479                             max_iterations=10):
480
481     # optimize parameter posterior
482     print("Updating posterior...")
--> 483     f =
↪ self.fit_posterior(n_sample=n_sample, lr=lr, max_epochs=max_epochs)
485     # init evidence, fail count, iteration count
486     # self.approx_evidence()
487     # print("\nlog evidence: {:.3f}\n".format(self.approx_evidence()))
488     # previdence = np.copy(self.log_evidence)
489     previdence = -np.inf
```

File g:

```
↪ \True_FYP\01_Project_Overview\Paper\Physics-constrainedneuralordinarydifferenialequationm
↪ py:452, in gLV.fit_posterior(self, n_sample, lr, beta1, beta2, epsilon,
↪ max_epochs, tol, patience)
441 # gradient of log likelihood
442 gradients = grad_neg_log_likelihood_lambda(self.shapes,
443                                             self.t[idx],
444                                             self.U[idx],
445                                             self.s_cap, self.m_cap,
446                                             self.lambda) / n_sample
--> 452 if any(np.isnan(gradients)) or any(np.isinf(gradients)):
453     n_nan += 1
455 # ignore exploding gradients
456 # setting the limit too low can cause fitting problems
```

KeyboardInterrupt:

```
[ ]: model.nu2
```

```
[ ]: array([0.0001      , 0.00417933, 0.00217222, 0.0001      , 0.00400857,  
          0.00559205, 0.00548172, 0.00353611, 0.0001      , 0.00343298,  
          0.0001      ])
```

```
[ ]: np.sqrt(model.nu2)
```

```
[ ]: array([0.01      , 0.06464775, 0.04660705, 0.01      , 0.06331326,  
          0.07477999, 0.07403863, 0.05946521, 0.01      , 0.05859164,  
          0.01      ])
```

```
[ ]: model.sigma2
```

```
[ ]: array([0.06178216, 0.02976103, 0.24702163, 0.09440023, 0.13129839,  
          0.2102742 , 0.23303026, 0.07934366, 0.23273022, 0.06317604,  
          0.07140832])
```

```
[ ]: # Define function to make predictions on test data  
def plot_sample(model, df_test):  
  
    # save true values and predictions  
    true = []  
    pred = []  
    spcs = []  
  
    # pull a random community trajectory  
    for treatment, comm_data in df_test.groupby("Treatments"):  
  
        # make sure comm_data is sorted in chronological order  
        comm_data.sort_values(by='Time', ascending=True, inplace=True)  
        tspan = comm_data.Time.values  
  
        # pull just the community data  
        species_true = comm_data[species].values  
        mediators_true = comm_data[mediators].values  
        input_vals = comm_data[inputs].values[0]  
  
        output_true = np.concatenate((species_true, mediators_true), -1)  
  
        # determine initial condition  
        s0 = species_true[0]  
        r0 = mediators_true[0]  
        x0 = np.concatenate((s0, r0))
```

```

# increase teval
t_eval = np.linspace(0, tspan[-1] + 5., 100)
steps = len(t_eval)

# generate posterior samples
species_pred, mediators_pred = model.predict_sample(x0,
                                                    t_eval,
                                                    n_sample=50,
                                                    inputs=input_vals)

# compute summary statistics (mean and variance) of posterior
# predictive distribution
E_species_pred, V_species_pred, E_mediators_pred, V_mediators_pred =
model.predict(x0,
              t_eval,
              n_sample=100,
              inputs=input_vals)

# concatenate species and mediator predictions
output_pred = np.concatenate((species_pred, mediators_pred), -1)
E_output_pred = np.concatenate((E_species_pred, E_mediators_pred), -1)
V_output_pred = np.concatenate((V_species_pred, V_mediators_pred), -1)

# predict end-point measured values
true.append(species_true[-1])
pred.append(species_pred[-1])
spcs.append(species)

# plot the results
plt.figure(figsize=(5, 5))
for i, sp in enumerate(species):
    out_true = output_true[:, i]
    plt.scatter(tspan, out_true, color='C{}'.format(i), edgecolor='k',
s=100, zorder=2, label=sp)

# plot posterior samples
for out in output_pred:
    plt.plot(t_eval, out[:, i], color='C{}'.format(i), alpha=.15,
zorder=0)

# plot posterior predictive as a Gaussian
plt.fill_between(t_eval,

```

```

        E_output_pred[:,i] - V_output_pred[:, i] ** .5,
        E_output_pred[:,i] + V_output_pred[:, i] ** .5,
        color='C{}'.format(i), alpha=0.2)

    # plot mean of posterior
    plt.plot(t_eval, E_output_pred[:,i], color='C{}'.format(i),
↳linewidth=3, zorder=1)

    plt.axhline(y=0, linestyle='-', color='k', alpha=.5)
    plt.xlabel("Time (hr)")
    plt.ylabel("Species abundance")
    # plt.title(input_vals)

    # plt.ylim([0, .5])
    plt.tight_layout()
    plt.show()

```

```
[ ]: # plot_sample(model, df)
```

```
[ ]: # function to make predictions on test set
def predict_df(model, df):

    # save measured and predicted values
    pred_dfs = []

    for exp_name, exp_df in df.groupby("Treatments"):

        # make sure comm_data is sorted in chronological order
        exp_df.sort_values(by='Time', ascending=True, inplace=True)

        # get initial condition and evaluation times
        t_span = exp_df.Time.values
        Y_m = exp_df[list(species) + list(mediators)].values
        inputs_true = exp_df[inputs].values

        # predict
        s_pred, s_var, m_pred, m_var = model.predict(Y_m, t_span,
↳inputs=inputs_true[0])

        # save dataframe with predictions
        pred_df = pd.DataFrame()
        pred_df["Treatments"] = [exp_name] * len(t_span)
        pred_df['Time'] = t_span

        # save mean
        pred_df[[f'E[{s}]' for s in species]] = s_pred
        pred_df[[f'E[{m}]' for m in mediators]] = m_pred

```

```

# save variance
pred_df[[f'V[{s}]' for s in species]] = s_var
pred_df[[f'V[{m}]' for m in mediators]] = m_var

pred_dfs.append(pred_df)

# concatenate list
pred_df = pd.concat(pred_dfs)

return pred_df

```

```
[ ]: pred_df = predict_df(model, df)
```

```
[ ]: pred_df
```

```
[ ]:
```

	Treatments	Time	E[BT]	E[BV]	E[CH]	E[BU]	E[CS]	\
0	BT-BU-BV-CA-CS-DP	0	0.001667	0.001667	0.0	0.001667	0.001667	
1	BT-BU-BV-CA-CS-DP	12	1.087506	0.326462	0.0	0.294174	0.108149	
2	BT-BU-BV-CA-CS-DP	24	1.298050	0.094237	0.0	0.114143	0.319882	
0	BT-BU-BV-CA-DP	0	0.002000	0.002000	0.0	0.002000	0.000000	
1	BT-BU-BV-CA-DP	12	1.076252	0.330920	0.0	0.324914	0.000000	
..	...	...	...	...	...	...	...	
4	MonocultureMS014_0	12	0.000000	0.000000	0.0	0.000000	0.000000	
5	MonocultureMS014_0	15	0.000000	0.000000	0.0	0.000000	0.000000	
6	MonocultureMS014_0	18	0.000000	0.000000	0.0	0.000000	0.000000	
7	MonocultureMS014_0	21	0.000000	0.000000	0.0	0.000000	0.000000	
8	MonocultureMS014_0	24	0.000000	0.000000	0.0	0.000000	0.000000	

	E[CA]	E[DP]	E[DSM]	...	V[BV]	V[CH]	V[BU]	V[CS]	\
0	0.001667	0.001667	0.0	...	0.004179	0.002172	0.000100	0.004009	
1	0.015654	0.016807	0.0	...	0.008710	0.002172	0.008724	0.005626	
2	0.088264	0.005602	0.0	...	0.006158	0.002172	0.002103	0.018990	
0	0.002000	0.002000	0.0	...	0.004179	0.002172	0.000100	0.004009	
1	0.017714	0.015354	0.0	...	0.008898	0.002172	0.010550	0.004009	
..	...	...	...	...	...	...	...	...	
4	0.000000	0.000000	0.0	...	0.004179	0.002172	0.000100	0.004009	
5	0.000000	0.000000	0.0	...	0.004179	0.002172	0.000100	0.004009	
6	0.000000	0.000000	0.0	...	0.004179	0.002172	0.000100	0.004009	
7	0.000000	0.000000	0.0	...	0.004179	0.002172	0.000100	0.004009	
8	0.000000	0.000000	0.0	...	0.004179	0.002172	0.000100	0.004009	

	V[CA]	V[DP]	V[DSM]	V[MS001]	V[MS008]	V[MS014]
0	0.005593	0.005482	0.003536	0.0001	0.003433	0.000100
1	0.005648	0.005569	0.003536	0.0001	0.003433	0.000100
2	0.007563	0.005530	0.003536	0.0001	0.003433	0.000100
0	0.005593	0.005483	0.003536	0.0001	0.003433	0.000100

```

1  0.005662  0.005556  0.003536    0.0001  0.003433  0.000100
..      ...      ...      ...      ...      ...
4  0.005592  0.005482  0.003536    0.0001  0.003433  0.021782
5  0.005592  0.005482  0.003536    0.0001  0.003433  0.022072
6  0.005592  0.005482  0.003536    0.0001  0.003433  0.022104
7  0.005592  0.005482  0.003536    0.0001  0.003433  0.022107
8  0.005592  0.005482  0.003536    0.0001  0.003433  0.022108

```

[626 rows x 24 columns]

```

[34]: correlation_list = []

# plot fit
for variable in species:

    measured_values = df.iloc[df.Time.values>0][variable].values
    predicted_values = pred_df.iloc[pred_df.Time.values>0][f'E[{variable}']].
↪values
    predicted_errors = pred_df.iloc[pred_df.Time.values>0][f'V[{variable}']].
↪values ** .5

    # Plot the parity plot
    plt.errorbar(measured_values, predicted_values, yerr=predicted_errors,
                  fmt='o', color='blue', ecolor='gray', markersize=8,
↪markeredgecolor='k', capsize=3)

    # Add line of perfect agreement
    plt.plot([min(measured_values), max(measured_values)],
              [min(measured_values), max(measured_values)],
              linestyle='--', color='red')

    # Calculate Pearson correlation coefficient
    correlation, _ = pearsonr(measured_values, predicted_values)

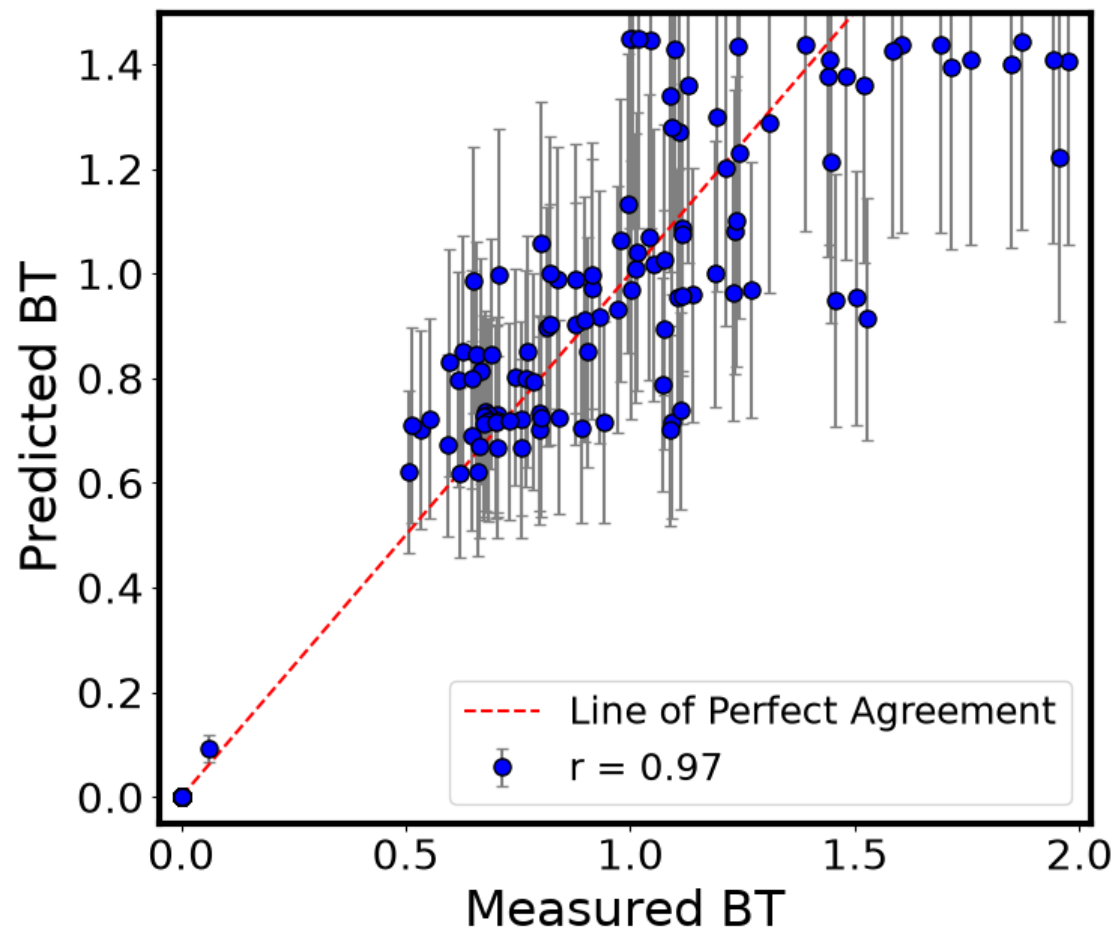
    # Add legend with Pearson correlation coefficient
    plt.legend(['Line of Perfect Agreement', f'r = {correlation:.2f}']) # ,
↪loc='upper left')

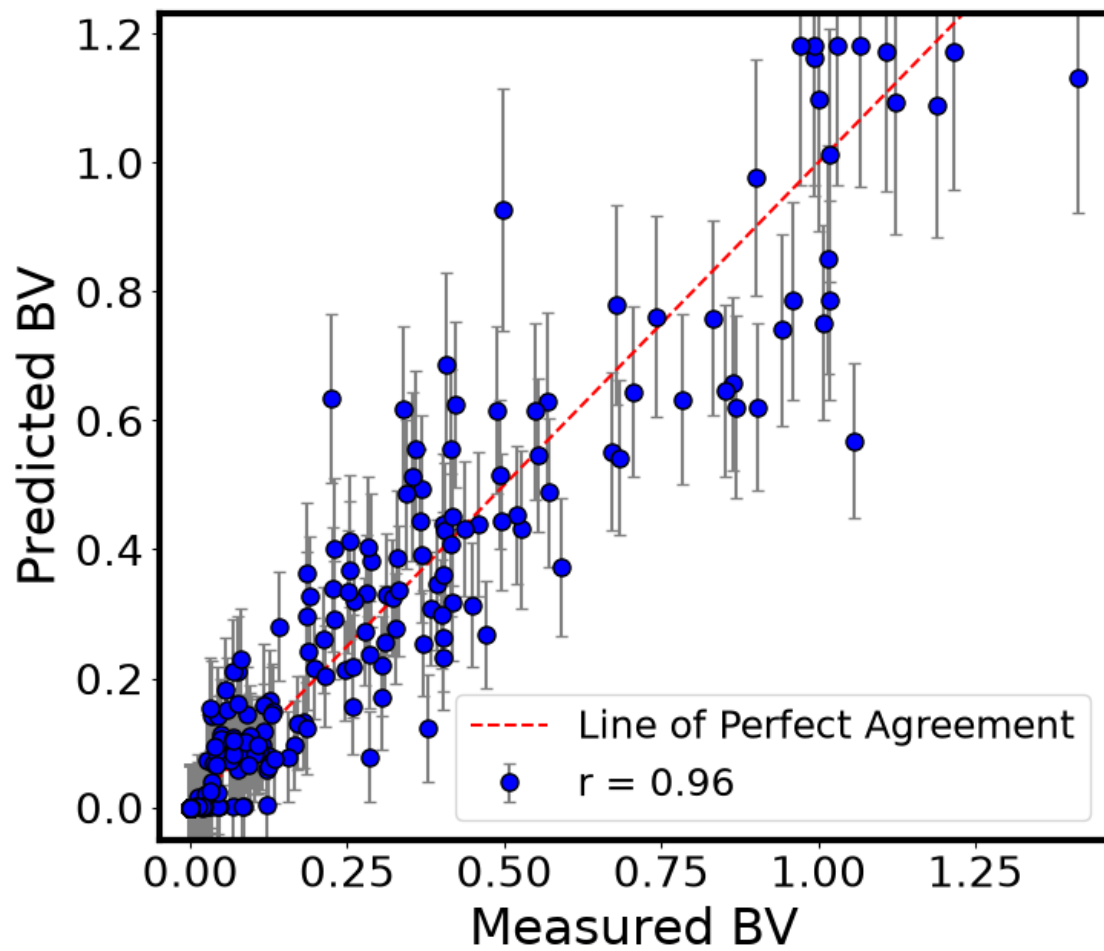
    # Set labels and title
    plt.xlabel(f'Measured {variable}')
    plt.ylabel(f'Predicted {variable}')

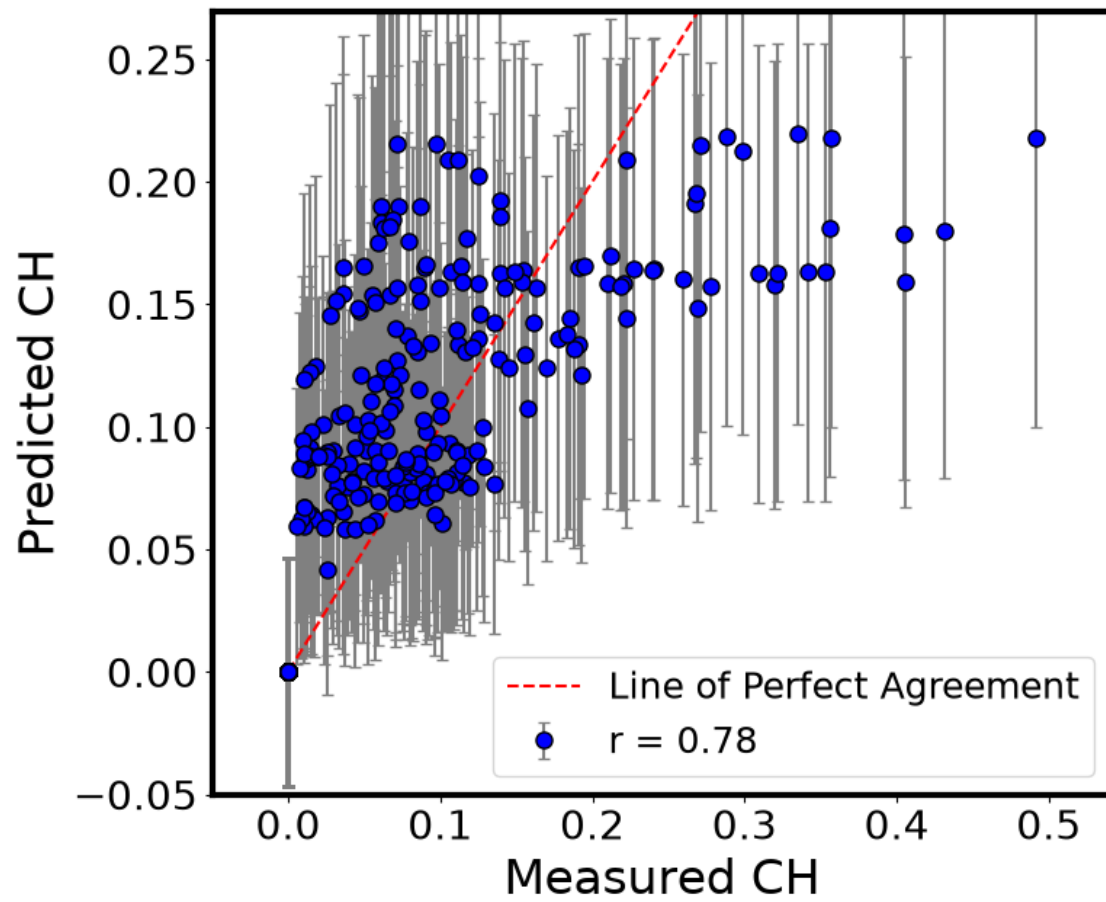
    # Set plot limits
    plt.xlim(min(measured_values)-0.05, max(measured_values)+0.05)
    plt.ylim(min(predicted_values)-0.05, max(predicted_values)+0.05)

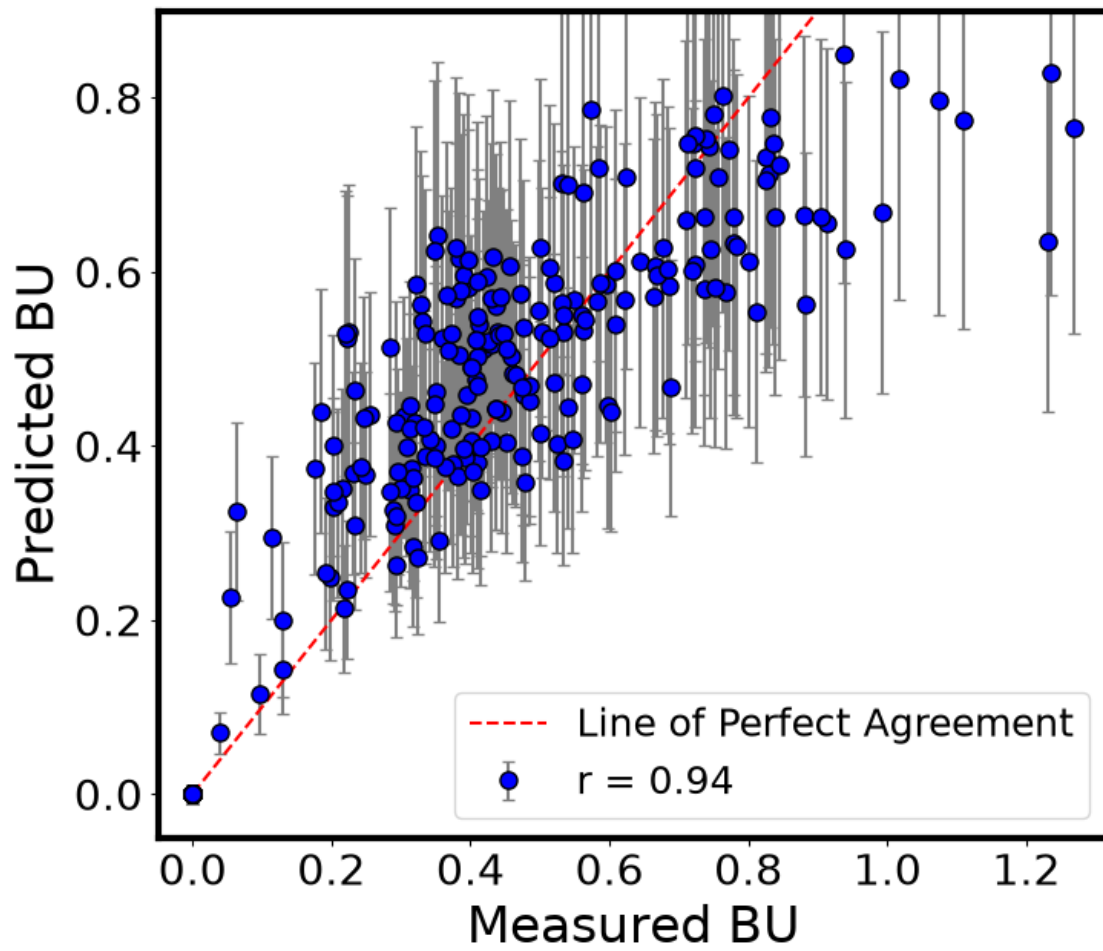
```

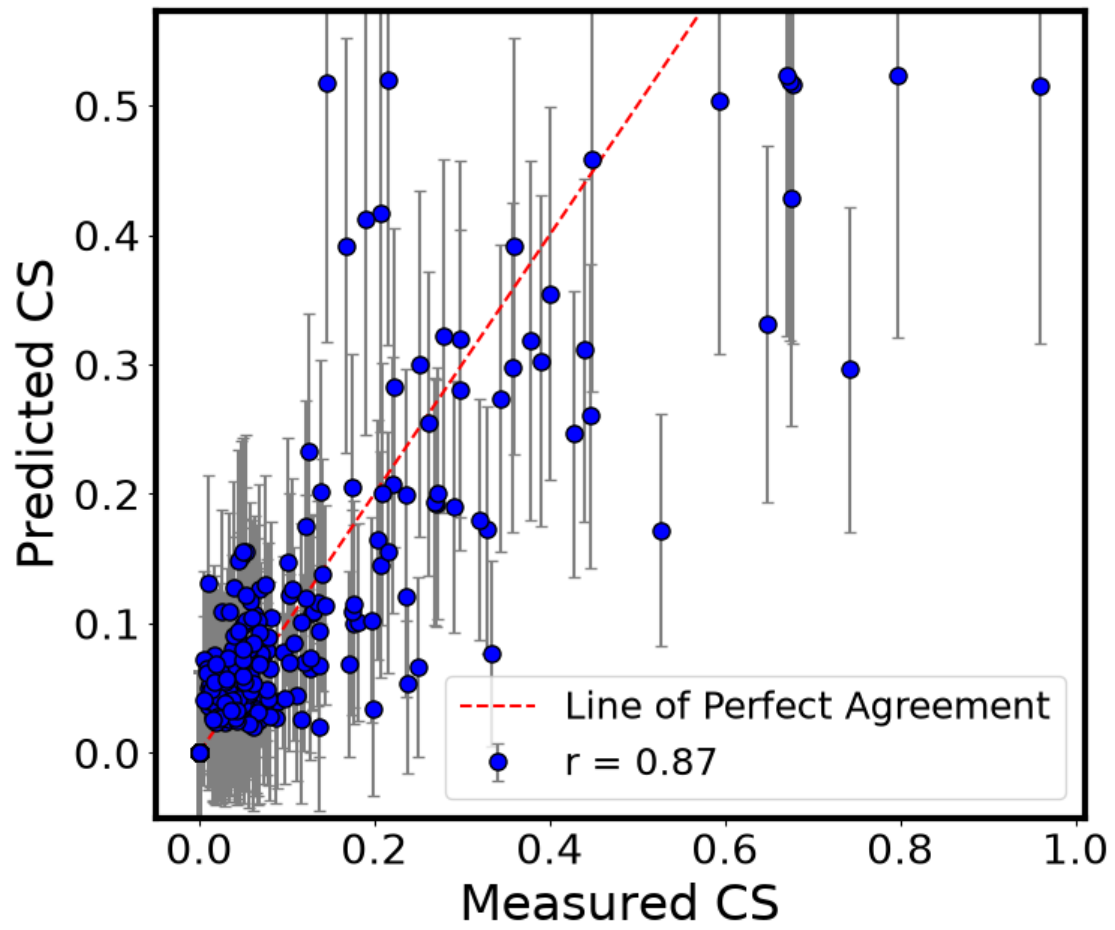
```
# Show the plot  
plt.show()  
  
# get R value  
correlation_list.append(correlation)
```

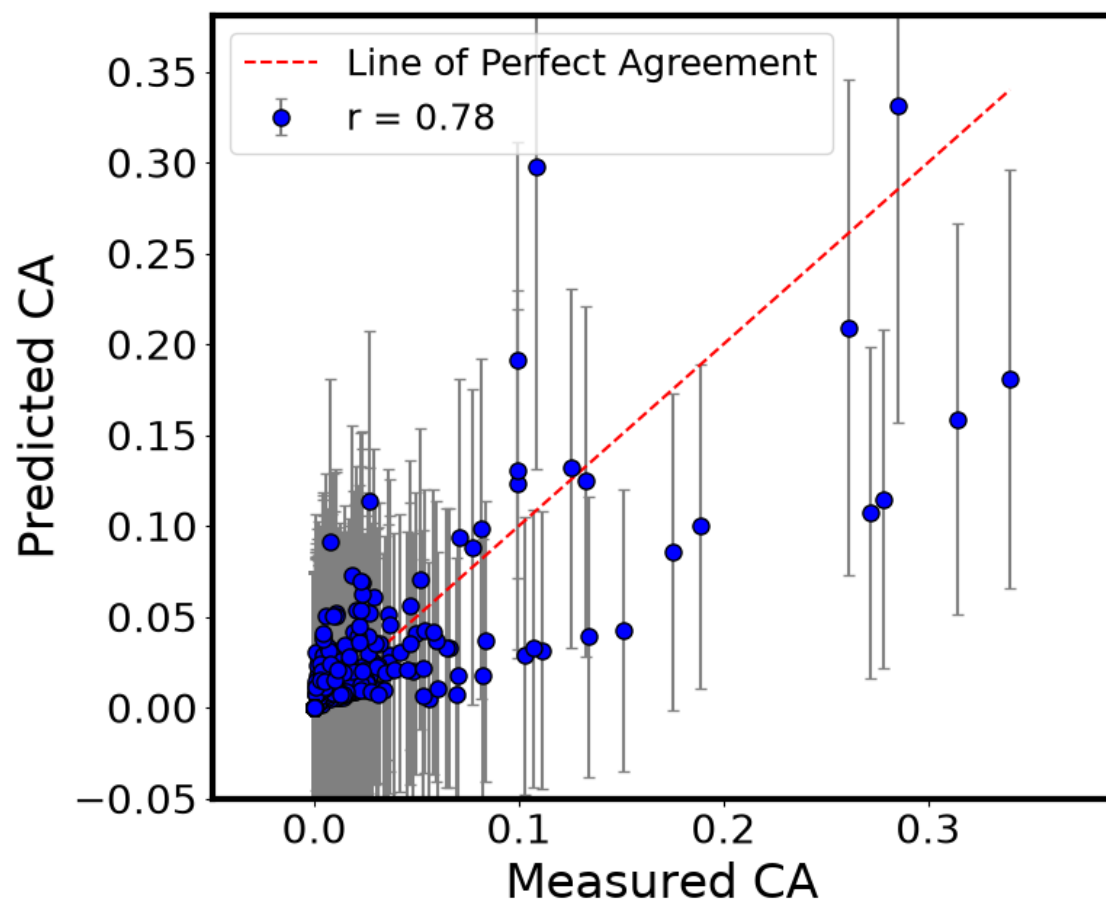


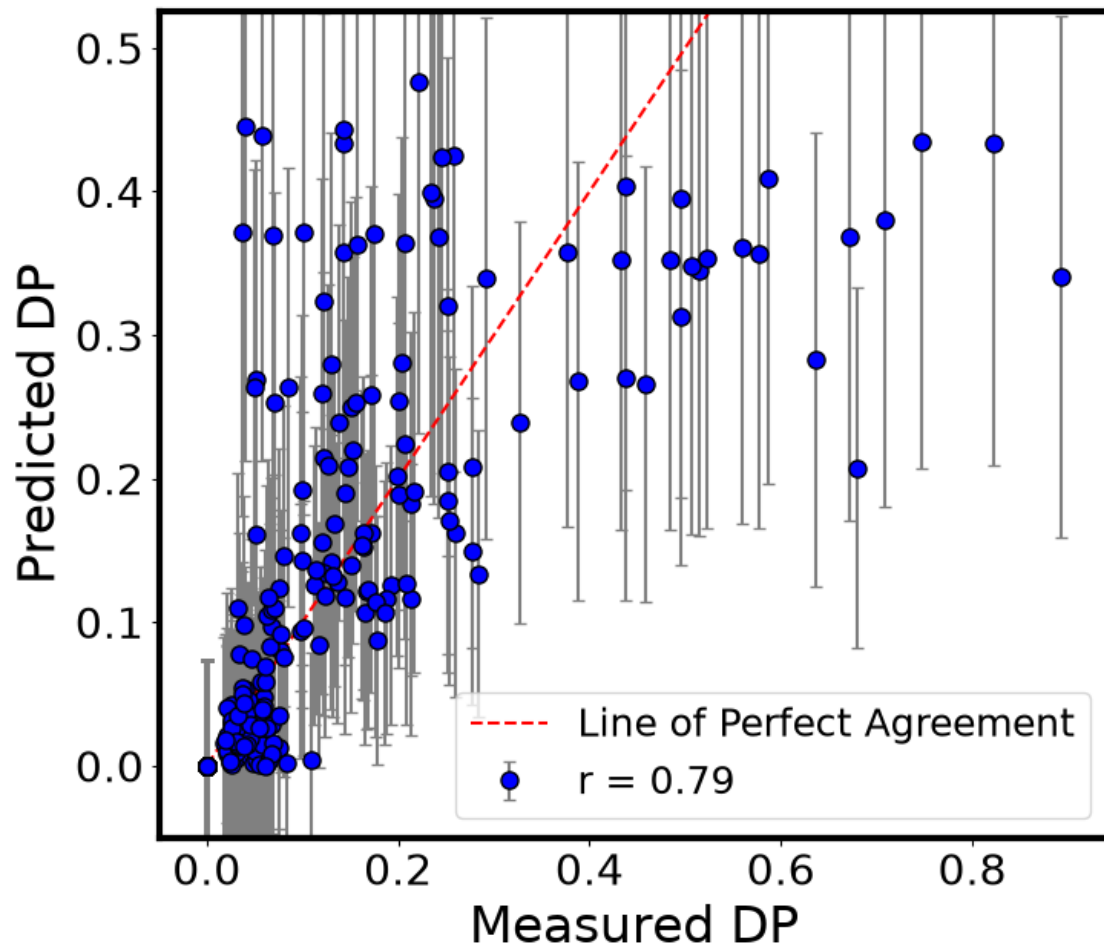


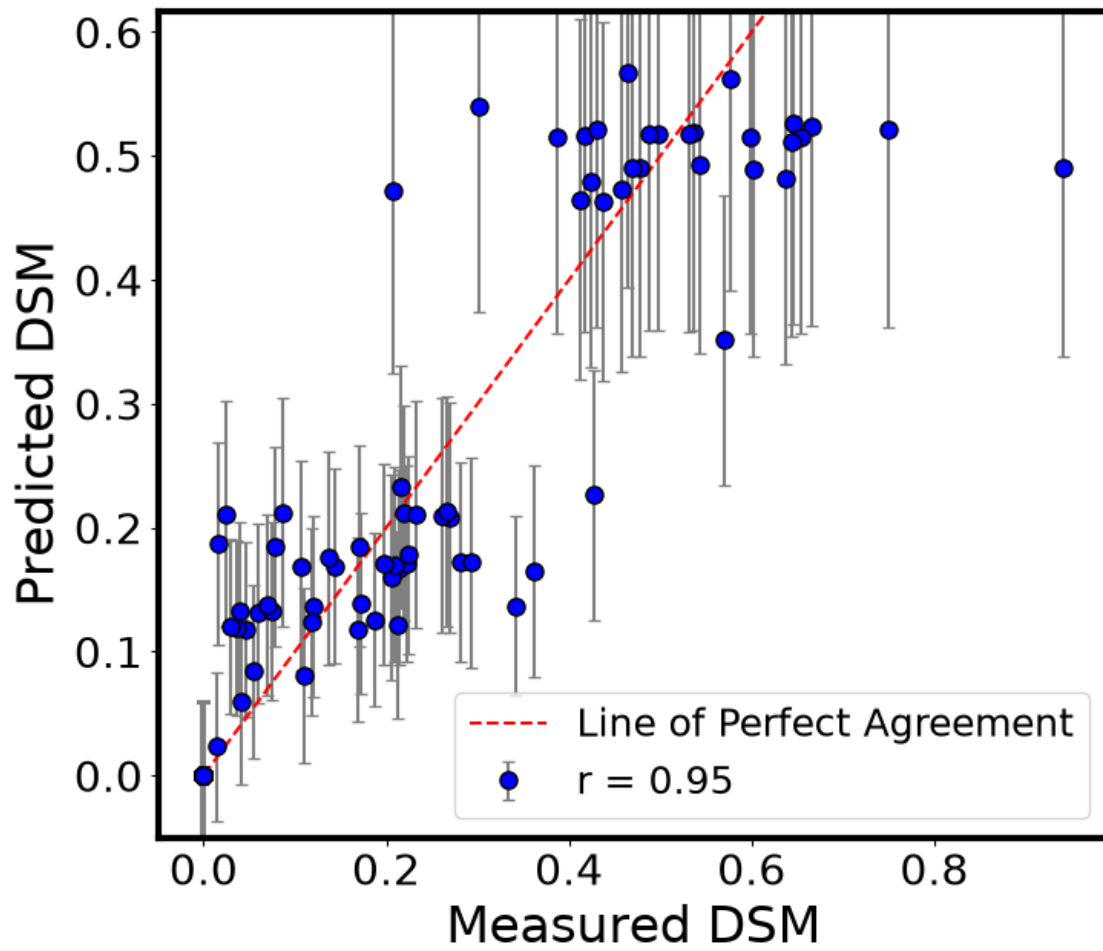


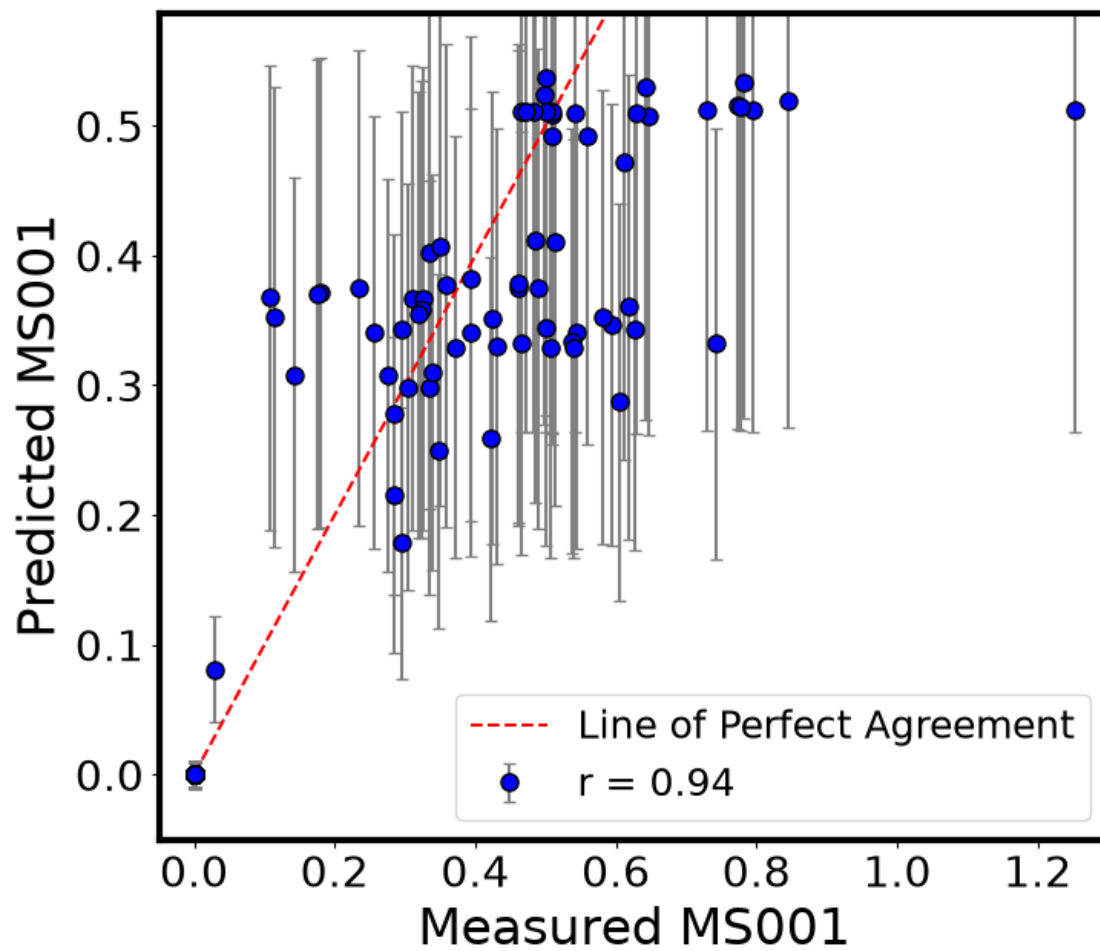


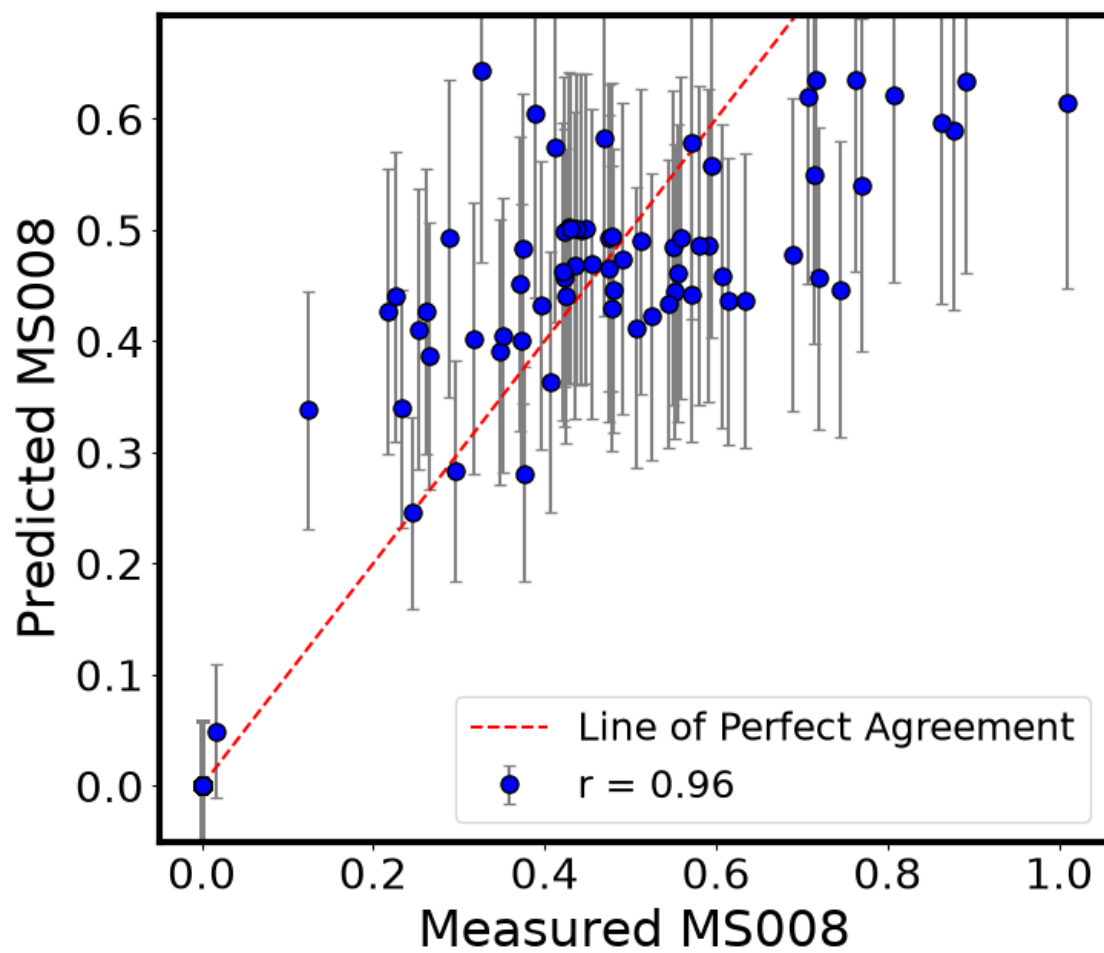


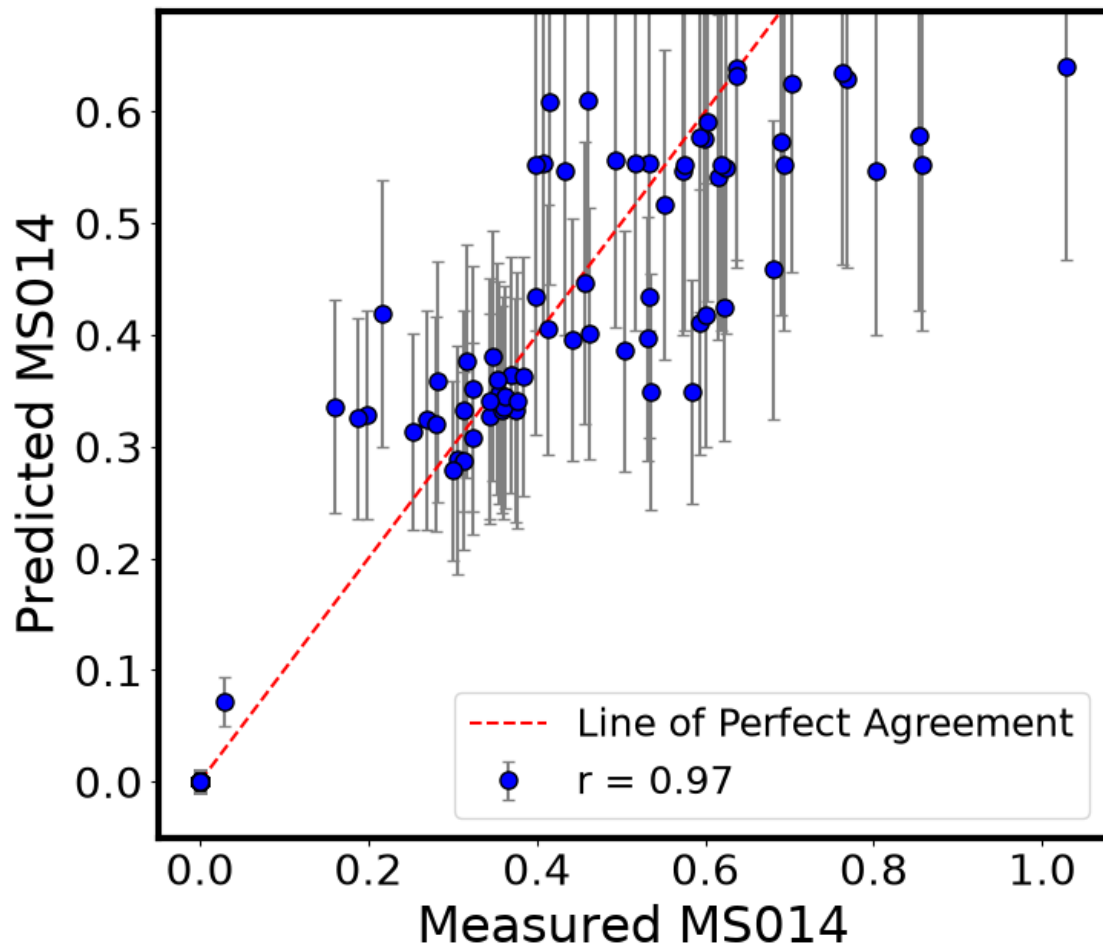












```
[ ]: # print(correlation_list)
df_cl = pd.DataFrame({'Value': correlation_list})
df_cl.to_csv('df_cl.csv', index=False, float_format='%.4f')
```

```
[np.float64(0.9708905698540337), np.float64(0.957225264329536),
 np.float64(0.779916055980105), np.float64(0.9386865523674732),
 np.float64(0.8690764230589348), np.float64(0.7832886727115237),
 np.float64(0.7883373223078977), np.float64(0.946908276418804),
 np.float64(0.9407696078019586), np.float64(0.957224972616591),
 np.float64(0.9693032683791056)]
```

```
[ ]:
```